

# SAS2Py User Manual

This manual is based on the current SAS2Py application behavior in the codebase as of May 4, 2026. Created by Chao Ji, PhD, email: [ugc8@cdc.gov](mailto:ugc8@cdc.gov).

## 1. Purpose

SAS2Py is designed to streamline the modernization of legacy SAS workflows by converting SAS programs into Python or R while preserving traceability and reviewability through AI (Artificial Intelligence) powered large language models. This app stores the original and converted code together, and keeps execution and review history for each saved conversion. For SAS users, this tool can be a valuable resource for learning how to write code in Python and R.

Features of SAS2Py include

- (1) SAS codes interpretation: when user inputs SAS codes, the tool will generate a plain-language interpretation of the SAS logic and a summary of the expected output for validation.
- (2) Automatically Convert SAS codes into Python or R.
- (3) Ran simple translated Python or R codes directly in the tool.
- (4) Upload files for validating translated code.
- (5) Iteratively refine AI-generated results using the enhancement feature.
- (6) Save projects for future access and use.

The current application includes these user-facing areas:

- **Dashboard:** Provides an overview of the user's activity, including the total number of conversions and reviews completed.
- **Studio:** Serves as the primary workspace for creating, converting, editing, and executing SAS code.
- **History:** Allows users to view past conversions and organize them by assigning them to specific projects.
- **Documentation:** Contains reference materials and guidance on how to use the application's features.
- **Settings:** Enables users to manage their profile information, such as updating their display name or email address.

## 2. Access and Sign-In

Authentication ensures that all user data - conversions, reviews, and execution results - are securely tied to a specific account.

### Supported methods:

- Email and password (standard authentication)
- Google OAuth (faster sign-in and SSO scenarios)

If you forget your password, you can use the **“Forgot Password”** option on the sign-in page to reset it. A password reset link will be sent to your registered email address, allowing you to create a new password.

### Once authenticated, users gain access to:

- Persistent storage of conversions and history
- Personalized dashboard metrics
- Execution environments (Python/R runners)
- Project organization features

## 3. Dashboard

The Dashboard provides a high-level overview of user activity.

Displayed metrics:

- Total stored conversions: Number of saved SAS-to-Python/R transformations
- Total reviews: Number of review entries across all conversions

This page is intentionally lightweight and serves as:

- A quick productivity snapshot
- A navigation starting point into Studio or History

Future enhancements may include trends, recent activity, or usage analytics.

## 4. Studio

The Studio is the core workspace where users perform all conversion, editing, and execution tasks. It is designed to support an end-to-end workflow:

- Input SAS code
- Convert to Python/R
- Analyze logic
- Refine output
- Execute and validate

- Provide a review and rating for the conversion

#### **4.1 Create a conversion**

Creating a conversion initializes a tracked unit of work.

Key inputs:

- Conversion name (required): Used for identification and search
- Target language: Python or R
- SAS source code: Manual input or '.sas' file upload

System validations ensure:

- No empty submissions
- Proper file formats
- Required metadata is present

Once created, the conversion becomes a persistent record in the system.

#### **4.2 Uploading a SAS file**

When a '.sas' file is uploaded:

- Its contents are loaded into the SAS editor.
- If the conversion name field is blank, the file name becomes the default conversion name.

#### **4.3 Conversion output**

After a successful conversion:

- The generated Python or R code appears in the output panel.
- The conversion is saved as a history entry.
- The original SAS code and converted code remain linked together.

### **5. SAS Analysis**

The Studio can generate an analysis of the SAS source code.

The current analysis returns:

- ``interpretation`` : a concise explanation of what the SAS program is doing
- ``expectedOutput`` : the expected output in business or data terms
- ``validationChecks`` : 3 to 6 checks that can be used to validate the conversion

This feature is useful when reviewing whether the converted output still matches the SAS program's intent.

## **6. Editing and Enhancing Converted Code**

After a conversion is created, users can continue working with the generated code.

### **6.1 Manual editing**

Users can edit the generated code directly in the Studio and save the changes back to the stored conversion.

### **6.2 AI enhancement**

Users can submit a refinement instruction, such as:

- optimize with vectorized pandas
- add typing
- improve performance
- clean up style

The application sends the current SAS source, the current converted code, and the enhancement instruction to the refinement service, then saves the updated output.

### **6.3 Downloading the result**

Users can download the current converted file:

- Python output downloads as `.py``
- R output downloads as `.R``

## **7. Running Converted Code**

The Studio supports execution of converted code.

Execution results can include:

- ``stdout``
- ``stderr``
- exit code
- timeout status
- duration in milliseconds
- detected packages
- policy mode
- generated plot images

### **7.1 Input files**

Users can attach one or more input files before execution.

The execution environment exposes uploaded files through ``SAS2PY_INPUT_DIR``.

Current runtime paths:

- Docker runs: ``/workspace/input``
- Databricks runs: temporary runtime directory created by the runner payload

## **7.2 Current backend behavior**

Based on the current code:

- Python execution defaults to Databricks unless another backend is explicitly requested or configured
- R execution defaults to Docker unless the deployment is configured differently

## **7.3 Plots**

If the executed code generates plots, SAS2Py captures them and displays them in the execution results panel.

## **7.4 Package policy**

Before execution, the application inspects imported packages.

Depending on deployment configuration, execution may be:

- allowed without restrictions
- blocked if a package is on the blocklist

- blocked if a package is not on the allowlist

If execution is rejected for package policy reasons, the user must modify the code or ask an administrator to change the execution policy.

## **7.5 File-size limits for Databricks execution**

When Databricks is used with uploaded execution files, large files may exceed notebook parameter limits.

If Azure Blob Storage handoff is not configured and the total uploaded file size exceeds the configured threshold, execution will fail with a size-limit error.

## **8. Reviews**

Each saved conversion can receive review notes.

Current review fields:

- Summary: optional
- Comments: required
- Rating: optional, must be between 1 and 5

Reviews are stored with the conversion and appear in the Studio view for that entry.

## **9. History and Projects**

The History page is used to manage saved conversions and organize them into projects.

## **9.1 Search**

Users can search conversions by:

- conversion name
- project name
- SAS code
- converted code

## **9.2 Projects**

Users can create projects with:

- name
- optional description

Conversions can be:

- assigned to a project
- moved between projects
- returned to the unassigned group

## **9.3 Deleting a project**

Deleting a project does not delete its conversions.



Current behavior:

- the project is removed
- its conversions are unassigned

#### **9.4 Deleting a conversion**

Deleting a conversion removes:

- the conversion entry
- its saved reviews
- its saved execution records

### **10. Settings**

The Settings page currently supports profile maintenance.

Users can:

- view their current display name
- view their email address
- update their display name

### **11. Current Validation and Error Rules**

The current implementation enforces these user-visible rules:

- Conversion name is required before conversion.
- SAS code is required before conversion.
- Review comments are required before saving a review.
- Ratings must be between 1 and 5.
- Code must exist before execution can start.
- Source upload must be a `.sas`` file.
- Users can only modify or delete conversions and projects they own.

## **12. Recommended User Workflow**

For typical use:

1. Sign in.
2. Create a named conversion in Studio.
3. Paste SAS code or upload a `.sas`` file.
4. Convert to Python or R.
5. Generate SAS analysis and compare the interpretation with the output.
6. Edit or enhance the converted code as needed.
7. Run the converted code with any required input files.
8. Review stdout, stderr, plots, and package detection.
9. Save review notes.
10. Organize the conversion into a project from the History page.

## **13. Notes for Administrators and Power Users**

Some behavior depends on deployment configuration rather than a visible user setting, including:

- OpenAI model selection
- execution backend configuration
- Databricks runner setup
- Docker runner setup
- package allowlist or blocklist policy
- Azure Blob Storage handoff for large Databricks input files

If users encounter execution, authentication, or storage-related errors, these settings should be checked in the deployment environment.